?? FlyShelf ? Microsoft Store Build & Compliance Guide

This guide explains the hybrid deployment architecture of FlyShelf, designed to achieve **100% Microsoft Store Compliance** while keeping the standalone (unpackaged) portable build extremely lightweight and **under 90 MB**.

?? The Compliance & Size Challenge

1. Microsoft Store Policy 10.2.1 (Security)
Microsoft Store App Policies strictly forbid applications from dynamically downloading and exec
uting arbitrary binaries at runtime (such as '.exe', '.dll', or '.bat' files). All executable c
ode must be included directly in the submitted MSIX package so Microsoft's scanners can scan an
d certify it during the ingestion process.
* **Store Impact**: If the app attempts to download 'cloudflared.exe' dynamically on first star
tup, it will be immediately flagged and rejected by Store certification.

2. Standalone Lightweight Constraint (< 90 MB)
A packaged 'cloudflared.exe' binary adds approximately ~45 MB to the application's distribution size.
* **Portable Impact**: To keep the direct standalone '.exe' download extremely lightweight and
fast (well under the 90 MB ceiling), we must **not** bundle 'cloudflared.exe' directly inside t
he standalone installer/EXE.

??? The Hybrid Solution: Conditional Bundling

To satisfy both conditions, FlyShelf uses a **conditional MSBuild compilation strategy** inside [FlyShelf.csproj](file:///e:/exeapps/FlyShelf/FlyShelf_PC/FlyShelf.csproj):

1. **Standalone Build**:
   - Compiled with 'StorePublish = false'.
   - 'cloudflared.exe' is **omitted** from the compiled executable.
   - At runtime, if global sync is turned on, the app securely downloads the verified agent on-
demand into '%AppData%\FlyShelf\agent\cloudflared.exe' and validates its SHA-256 signature befo
re running.
2. **Microsoft Store Build**:
   - Compiled with 'StorePublish = true'.
   - The build pipeline conditionally links and packages 'MicrosoftBuild\agent\cloudflared.exe'
 into the MSIX package.
   - At runtime, 'CloudflareDaemon.cs' detects that the app is packaged ('IsPackaged() == true'
) and skips any dynamic download attempts entirely. Instead, it runs the secure agent pre-bundl
ed in the package root.

?? How to Build the Store Package

Step 1: Obtain the Verified Secure Agent
1. Download 'cloudflared-windows-amd64.exe' (v2024.12.2) from Cloudflare's official releases.

2. Verify its cryptographic hash:
   - **Expected SHA-256**: ‘c2f4a3c3ea4c62eed562ede027d586a6044d35517e335e642f4e9783e651e4a3‘
3. Rename the binary to ‘cloudflared.exe‘.
4. Create the folder ‘MicrosoftBuild\agent\‘ if it does not exist, and place the executable the
re:
   - **Target Path**: ‘E:\exeapps\FlyShelf\MicrosoftBuild\agent\cloudflared.exe‘

Step 2: Run the Store Build Pipeline
1. Open a command prompt or PowerShell window in the ‘MicrosoftBuild‘ folder.
2. Execute the automated build pipeline script:
   ‘‘‘cmd
   .\Build_Store.bat
   ‘‘‘
3. The script will:
   - Stage the assets and manifest.
   - Inject the ‘MSIX_STORE‘ define flag.
   - Retrieve ‘cloudflared.exe‘ from ‘MicrosoftBuild\agent\cloudflared.exe‘ and bundle it insid
e the MSIX package.
   - Generate the final ‘.msix‘ container inside ‘MicrosoftBuild\Output\‘.

Step 3: Test and Submit
1. Double-click the generated ‘.msix‘ file in the ‘Output‘ folder to install and run the packag
ed app locally.
2. Turn on Global Web Sync in the settings tab and verify that the tunnel starts instantly (you
 will see the secure Cloudflare ‘*.trycloudflare.com‘ URL appear). Check the logs to ensure **n
o** downloads were performed.
3. Sign in to the [Microsoft Partner Center](https://partner.microsoft.com).
4. Upload the generated ‘.msix‘ file to your app submission page and submit for certification.